

gHostMail - Secure Ephemeral Communication

Project Type: Student-led

Background

Standard email is convenient but gives senders little control after delivery, while disposable inboxes emphasize anonymity more than confidentiality. gHostMail is a web application that uses ordinary email only for notifications; sensitive messages and attachments remain inside temporary secure/encrypted rooms. Encryption occurs in the sender's browser before the upload, recipients can then reply inside the same protected room, and each room clearly explains its security guarantees and limitations. The main goal is not to replace Gmail or Outlook, but to add a usable secure layer for short-lived exchanges with users of any email provider.

Project Needs

1. Create a browser-based client that encrypts message content and attachments before they leave the sender's device.
2. Build a seamless recipient experience with strong email verification, a higher-security one-time code, and trusted-device modes.
3. Develop a lifecycle engine supporting timed expiration, one-view access, sender revocation, failed-attempt limits, and cryptographic erasure.
4. Produce a documented threat model and automated tests proving that stored ciphertext cannot be read, altered undetected, or accessed after expiration.

Project Scope

1. True end-to-end encryption applies to content composed inside gHostMail; conventional email carries only a notification and opaque link.
2. The core product includes encrypted text, attachments, two-way replies, expiration policies, recipient authentication, and a clear security summary for every room.
3. The system cannot prevent an authorized recipient from copying decrypted content. Browser extensions, multi-recipient permissions, and metadata padding are stretch goals.

Potential Technical Considerations

Front End: React and TypeScript with the Web Crypto API for authenticated client-side encryption, plus accessible mobile and desktop layouts.

Back End: Node.js or Python APIs, PostgreSQL for minimal metadata, encrypted object storage, a reliable expiration worker, and provider-independent email delivery.

Identity and Keys: Passkeys for authentication, recipient public keys or separately shared one-time codes for unlocking, and audited cryptographic libraries rather than custom algorithms.

Security and Reliability: Rate limiting, tamper detection, secret-free logs, integration/load testing, recovery from interrupted deletion jobs, and SPF/DKIM/DMARC configuration.

Deliverables

1. A deployed web application for creating, receiving, replying to, revoking, and expiring secure rooms with encrypted attachments.
2. Working notification delivery and recipient access across major email providers, with a browser extension as a stretch deliverable.
3. A threat model, protocol specification, automated adversarial test suite, performance results, and usability evaluation.
4. A live demonstration showing ciphertext-only storage, tamper rejection, authorized decryption, one-view expiration, and failure to recover content from an expired database snapshot.

Assumptions and Dependencies

TBD